Documento de Engenharia de Software

Feature Request & Product Roadmap Management System

EDS-14 — Modelo Entidade-Relação

Ano Letivo 2026/27

Modelo Entidade-Relação (PostgreSQL)

O modelo foi derivado do diagrama de classes (EDS-13): cada classe deu origem a uma tabela, cada ligação de um para muitos a uma chave estrangeira (FK) e cada ligação de muitos para muitos a uma tabela de associação (`equipa_membro`, `pedido_tag` e `dependencia`).

Como ler o diagrama:

- `PK` é a chave primária e `FK` a chave estrangeira; o ponto antes de uma coluna indica que é obrigatória (`NOT NULL`).
- Nas pontas das linhas (notação pé-de-galinha): dois traços significam "exatamente um", círculo e traço "zero ou um", círculo e pé-de-galinha "zero ou mais", traço e pé-de-galinha "um ou mais".
- Os enumerados do diagrama de classes (perfil e estados) são guardados como `VARCHAR` com valores fixos.

Regras garantidas pela base de dados:

- O `voto` tem chave primária composta (`utilizador_id`, `pedido_id`), o que impede votos repetidos (US 2.2).
- O `comentario` tem um `CHECK` que obriga a preencher exatamente um de `pedido_id` ou `user_story_id`.
- O `email` do utilizador e os nomes da categoria e da tag são únicos; a `pedido_tag` tem chave primária composta (`pedido_id`, `tag_id`).
- O `pedido_id` da `user_story` é opcional: uma story criada manualmente não tem pedido de origem (US 3.1).
- A `data_prevista` da `user_story` é usada nos alertas de dependência em risco e bloqueada (US 3.3).
- As sugestões da IA ficam na coluna `sugestoes_ia` (`JSONB`) do pedido e a justificação de uma rejeição em `justificacao_rejeicao`.
- Uma User Story pode ter zero ou mais critérios de aceitação; a exigência de pelo menos um (Definition of Ready) é verificada pela aplicação quando a story entra num Sprint.
- A `ordem_backlog` é obrigatória e é preenchida no momento em que a User Story é criada (último lugar do backlog).
- O `criterio_aceitacao` e o `anexo` são apagados em cascata com a User Story e o pedido a que pertencem.

O diagrama está na página seguinte, em formato vetorial e numa página maior do que A4: pode ser ampliado sem perder qualidade. O código PlantUML que o gera está nas últimas páginas e pode ser colado em `planttext.com` para o editar.

Modelo Entidade-Relação (PostgreSQL)

utilizador
- id : SERIAL «PK»
- nome : VARCHAR(100)
- email : VARCHAR(150) «UNIQUE»
- password_hash : VARCHAR(255)
- perfil : VARCHAR(20)
- capacidade_sp : INTEGER

equipa
- id : SERIAL «PK»
- nome : VARCHAR(100)

categoria
- id : SERIAL «PK»
- nome : VARCHAR(60) «UNIQUE»

release
- id : SERIAL «PK»
- nome : VARCHAR(100)
- data_prevista : DATE

sprint
- id : SERIAL «PK»
- equipa_id : INTEGER «FK»
- numero : INTEGER
- objetivo : TEXT
- data_inicio : DATE
- data_fim : DATE

equipa_membro
- equipa_id : INTEGER «PK, FK»
- utilizador_id : INTEGER «PK, FK»

tag
- id : SERIAL «PK»
- nome : VARCHAR(60) «UNIQUE»

notificacao
- id : SERIAL «PK»
- utilizador_id : INTEGER «FK»
- mensagem : TEXT
- lida : BOOLEAN
- data : TIMESTAMP

pedido
- id : SERIAL «PK»
- autor_id : INTEGER «FK»
- categoria_id : INTEGER «FK»
- release_id : INTEGER «FK»
- titulo : VARCHAR(200)
- problema : TEXT
- estado : VARCHAR(20)
- esforco : SMALLINT
- impacto : SMALLINT
- data_submissao : TIMESTAMP
- data_lancamento : TIMESTAMP
- justificacao_rejeicao : TEXT
- sugestoes_ia : JSONB

milestone
- id : SERIAL «PK»
- release_id : INTEGER «FK»
- nome : VARCHAR(100)
- data : DATE

user_story
- id : SERIAL «PK»
- pedido_id : INTEGER «FK»
- sprint_id : INTEGER «FK»
- responsavel_id : INTEGER «FK»
- descricao : TEXT
- story_points : SMALLINT
- ordem_backlog : INTEGER
- estado : VARCHAR(20)
- data_prevista : DATE
- data_inicio : TIMESTAMP
- data_conclusao : TIMESTAMP

pedido_tag
- pedido_id : INTEGER «PK, FK»
- tag_id : INTEGER «PK, FK»

anexo
- id : SERIAL «PK»
- pedido_id : INTEGER «FK»
- nome_ficheiro : VARCHAR(255)
- url : VARCHAR(500)

voto
- utilizador_id : INTEGER «PK, FK»
- pedido_id : INTEGER «PK, FK»
- data : TIMESTAMP

dependencia
- user_story_id : INTEGER «PK, FK»
- depende_de_id : INTEGER «PK, FK»

criterio_aceitacao
- id : SERIAL «PK»
- user_story_id : INTEGER «FK»
- dado : TEXT
- quando : TEXT
- entao : TEXT

comentario
- id : SERIAL «PK»
- autor_id : INTEGER «FK»
- pedido_id : INTEGER «FK»
- user_story_id : INTEGER «FK»
- texto : TEXT
- data : TIMESTAMP

PK composta (utilizador_id, pedido_id): um voto por utilizador e pedido

CHECK: exatamente um de pedido_id / user_story_id preenchido

Código PlantUML

```
@startuml modelo_er_v
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 16
skinparam dpi 96
skinparam linetype ortho
skinparam nodesep 60
skinparam ranksep 90
hide circle
skinparam class {
  BackgroundColor White
  BorderColor #111111
  FontStyle bold
  ArrowColor #222222
  ArrowThickness 1.6
}
title Modelo Entidade-Relação (PostgreSQL)

entity "utilizador" as utilizador #E8F0FE {
  * id : SERIAL <<PK>>
  --
  * nome : VARCHAR(100)
  * email : VARCHAR(150) <<UNIQUE>>
  * password_hash : VARCHAR(255)
  * perfil : VARCHAR(20)
  capacidade_sp : INTEGER
}
entity "equipa" as equipa #E8F0FE {
  * id : SERIAL <<PK>>
  --
  * nome : VARCHAR(100)
}
entity "equipa_membro" as equipa_membro #E8F0FE {
  * equipa_id : INTEGER <<PK, FK>>
  * utilizador_id : INTEGER <<PK, FK>>
}
entity "notificacao" as notificacao #E8F0FE {
  * id : SERIAL <<PK>>
  --
  * utilizador_id : INTEGER <<FK>>
  * mensagem : TEXT
  * lida : BOOLEAN
  * data : TIMESTAMP
}

entity "categoria" as categoria #FFF4E0 {
  * id : SERIAL <<PK>>
  --
  * nome : VARCHAR(60) <<UNIQUE>>
}
entity "pedido" as pedido #FFF4E0 {
  * id : SERIAL <<PK>>
  --
  * autor_id : INTEGER <<FK>>
  categoria_id : INTEGER <<FK>>
  release_id : INTEGER <<FK>>
  * titulo : VARCHAR(200)
  * problema : TEXT
  * estado : VARCHAR(20)
  esforco : SMALLINT
  impacto : SMALLINT
```

```
  * data_submissao : TIMESTAMP
  data_lancamento : TIMESTAMP
  justificacao_rejeicao : TEXT
  sugestoes_ia : JSONB
}
entity "tag" as tag #FFF4E0 {
  * id : SERIAL <<PK>>
  --
  * nome : VARCHAR(60) <<UNIQUE>>
}
entity "pedido_tag" as pedido_tag #FFF4E0 {
  * pedido_id : INTEGER <<PK, FK>>
  * tag_id : INTEGER <<PK, FK>>
}
entity "anexo" as anexo #FFF4E0 {
  * id : SERIAL <<PK>>
  --
  * pedido_id : INTEGER <<FK>>
  * nome_ficheiro : VARCHAR(255)
  * url : VARCHAR(500)
}
entity "voto" as voto #FFF4E0 {
  * utilizador_id : INTEGER <<PK, FK>>
  * pedido_id : INTEGER <<PK, FK>>
  --
  * data : TIMESTAMP
}
entity "comentario" as comentario #FFF4E0 {
  * id : SERIAL <<PK>>
  --
  * autor_id : INTEGER <<FK>>
  pedido_id : INTEGER <<FK>>
  user_story_id : INTEGER <<FK>>
  * texto : TEXT
  * data : TIMESTAMP
}

entity "sprint" as sprint #E6F4EA {
  * id : SERIAL <<PK>>
  --
  * equipa_id : INTEGER <<FK>>
  * numero : INTEGER
  objetivo : TEXT
  * data_inicio : DATE
  * data_fim : DATE
}
entity "user_story" as user_story #E6F4EA {
  * id : SERIAL <<PK>>
  --
  pedido_id : INTEGER <<FK>>
  sprint_id : INTEGER <<FK>>
  responsavel_id : INTEGER <<FK>>
  * descricao : TEXT
  story_points : SMALLINT
  * ordem_backlog : INTEGER
  * estado : VARCHAR(20)
  data_prevista : DATE
  data_inicio : TIMESTAMP
  data_conclusao : TIMESTAMP
}
entity "criterio_aceitacao" as criterio #E6F4EA {
  * id : SERIAL <<PK>>
  --
```

```
  * user_story_id : INTEGER <<FK>>
  * dado : TEXT
  * quando : TEXT
  * entao : TEXT
}
entity "dependencia" as dependencia #E6F4EA {
  * user_story_id : INTEGER <<PK, FK>>
  * depende_de_id : INTEGER <<PK, FK>>
}

entity "release" as release #F3E8FD {
  * id : SERIAL <<PK>>
  --
  * nome : VARCHAR(100)
  * data_prevista : DATE
}
entity "milestone" as milestone #F3E8FD {
  * id : SERIAL <<PK>>
  --
  * release_id : INTEGER <<FK>>
  * nome : VARCHAR(100)
  * data : DATE
}

' --- relações (pé-de-galinha)
equipa ||--o{ equipa_membro
utilizador ||--o{ equipa_membro
utilizador ||--o{ notificacao
utilizador ||--o{ pedido
categoria |o--o{ pedido
release |o--o{ pedido
pedido ||--o{ anexo
pedido ||--o{ pedido_tag
tag ||--o{ pedido_tag
utilizador ||--o{ voto
pedido ||--o{ voto
utilizador ||--o{ comentario
pedido |o--o{ comentario
user_story |o--o{ comentario
pedido |o--o{ user_story
sprint |o--o{ user_story
utilizador |o--o{ user_story
equipa ||--o{ sprint
user_story ||--o{ criterio
user_story ||--o{ dependencia
user_story ||--o{ dependencia
release ||--o{ milestone

note bottom of comentario
  CHECK: exatamente um de
  pedido_id / user_story_id preenchido
end note
note bottom of voto
  PK composta (utilizador_id, pedido_id):
  um voto por utilizador e pedido
end note
@enduml
```